Proposal for Final Project

Statistical Programming Paradigms and Workflows (BSPH 140.777)

Team: The Masons

1. Title and team

Working project title: Choosing a Hospital: A Patient-Facing Shiny App for Comparing Condition-Specific Outcomes.

Team: The Masons.
Team members: Sankalp (Sunny) Kumar, Wanting Shan, Shiyi Zhang, Yutong (Benny) Li, Mason Cleveland
Team leader: Sankalp Kumar

2. Question and audience

Our guiding question is: **How can patients and caregivers compare hospitals for a selected condition using publicly available measures of mortality, readmissions, cost and overall quality?**

Choosing a hospital involves interpreting information that may be difficult to find or compare. A hospital's overall rating does not necessarily describe its performance for a particular condition. Patients may also encounter missing measures, different reporting periods, and historical financial information without understanding how these affect a comparison.

We propose a Shiny application for patients and caregivers who are exploring options for planned care or discussing hospital choices with a clinician. Users will select a condition and location, inspect available hospital outcomes, and compare measures alongside their definitions and reporting periods. The initial version will focus on Maryland hospitals and three conditions: heart attack, heart failure, and pneumonia.

Our motivation is to allow patients to exercise their rights to choose health care services. The app will support informed comparisons rather than estimate an individual's disease risk or identify a universally best hospital. A patient's clinical needs, insurance network, specialist availability, and travel constraints remain relevant but are not fully captured in these data.

Historical Medicare payments will be considered as a separate exploratory feature rather than a measure of current affordability.

3. Related work

Medicare Care Compare already provides a public interface for comparing healthcare providers, including hospitals. The CMS Provider Data Catalog supplies the underlying public datasets that can support reproducible analyses and alternative interfaces.

Our project will build on these resources rather than introduce a new source of hospital quality measurement. Its intended contribution is a focused, condition-first comparison workflow for a small set of outcomes, with explicit explanations of missing data, reporting periods, and the distinction between overall hospital ratings and condition-specific performance. We will review Care Compare's existing hospital interface during development to refine which aspects our app can make easier to understand. Any claimed usability improvement will remain a design goal unless supported by user feedback.

CMS documentation also informs interpretation. Its complications and deaths guidance explains the meaning of mortality measures, while its unplanned hospital visits guidance distinguishes readmissions from other return-visit outcomes. We will use these definitions when creating patient-facing labels.

4. Data and access

Sources and available measures

We will acquire data through the **CMS Provider Data Catalog API**, which allows public requests without an API key. This API uses a different endpoint structure from the general data.cms.gov API. Four dataset endpoints have already been accessed successfully:

Dataset	Dataset ID	Intended use
Hospital General Information	`xubh-q36u`	Hospital identity, location, type, and overall stars
Complications and Deaths - Hospital	`ynj2-r877`	Condition-specific mortality and complication measures
Unplanned Hospital Visits - Hospital	`632h-zaca`	Condition-specific readmissions and other unplanned visits
Historical Payment and Value of Care - Hospital	`c7us-v4mf`	Optional, explicitly historical Medicare payment context

The outcome tables contain measure identifiers and names, scores, denominators, national comparison categories, uncertainty estimates where available, footnotes, and reporting dates. Hospital information can be linked to outcomes using the CMS facility ID. These are hospital-level aggregate records, not individual patient data.

Collection and preparation workflow

We will use reusable R functions to send state-filtered requests, retrieve successive pages using `limit` and `offset`, retry transient failures, and compare downloaded row counts with API totals. The preliminary workflow uses pages of 500 records. We will retain downloaded responses and collection metadata so that data displayed in the app can be traced to a source and retrieval date.

Preparation will preserve facility identifiers as text, filter outcome tables to the intended measure IDs, and join records by hospital and condition. We will retain original score strings and footnotes while creating numeric fields for analysis. Missing or suppressed scores will remain missing rather than being converted to zero. We will inspect key uniqueness and measure availability before producing comparison tables.

Access and interpretation limitations

Public access does not guarantee uninterrupted service or stable schemas. We will cache a dated extract for the deployed app and provide code to refresh it, rather than make an API call for every user interaction. Measurement periods may be older than the download date and may differ between measures; both will be documented.

Some hospitals lack reportable values or uncertainty estimates. The population represented can differ by measure, and Medicare-based estimates may not generalize to every patient. Reported adjusted rates should not be treated as individual risk predictions or used to reconstruct exact event counts from denominators.

Payments require a separate limitation. CMS's hospital data dictionary states that disease-specific Payment and Value of Care measures were removed from reporting beginning in July 2025. The historical endpoint remains accessible, and the sample contains older reporting windows. These values describe Medicare episode payments, not current prices, insurance-specific estimates, or patient out-of-pocket costs. If included, they will appear in a clearly dated historical view and will not contribute to a current recommendation score.

5. Planned work

We will organize development around the following tasks:

1. **Confirm the initial measures.**
2. **Build a reproducible data pipeline.**
3. **Develop transparent comparisons.**
4. **Build the Shiny interface.**
5. **Verify and deploy the product.** We will begin with measure-by-measure comparisons rather than a composite ranking. A weighted score would require additional decisions about scale, weights, missingness, and uncertainty; it is a possible extension only after the core comparison tool works. Machine learning is not necessary for the proposed question.

6. Programming paradigms

In addition to meaningful API collection, we plan to integrate **functional programming** and **command-line programming**.

Functional programming will structure acquisition and transformation. A reusable API function will be mapped over named dataset identifiers using `purrr::imap()`. Similar functions will select measures, parse scores, and summarize availability across conditions. This approach makes the workflow consistent across datasets and reduces repeated code. The preliminary R acquisition script already uses this pattern.

Command-line programming will provide a reproducible workflow from acquisition to reporting. We will extend the existing shell entry point into a script that accepts a state argument, runs the R data pipeline, performs validation, and renders the Quarto data report. The script will stop when a required stage fails and use documented relative paths. This gives collaborators one entry point for refreshing the app's cached data and checking the resulting outputs.

The components will work together as follows: a command-line entry point runs functional R acquisition and cleaning routines, which create validated data for Shiny and a rendered Quarto report. We will discuss whether the planned command-line component is sufficiently substantive at the proposal meeting. Using several software packages or languages alone will not be presented as evidence of multiple paradigms.

Machine learning will potentially play a role in this project, depending on time, data quality, and feasibility.

7. Software

The main implementation will use **R**:

- `httr2` for API requests, response handling, and retries;
- `dplyr`, `tidyr`, and `readr` for preparation and CSV input/output;

- `purrr` for mapping reusable functions over datasets and conditions;
- `ggplot2` for descriptive comparison graphics;
- `shiny` for the interactive application;
- `knitr` and Quarto for executable documentation and PDF reports.

A Bash script will coordinate the command-line workflow. Git and a shared GitHub repository will support collaboration, review, and contribution tracking. We plan to deploy through shinyapps.io, subject to account access and hosting limits, and will provide local run instructions as well. We will record package versions and document dependencies, with `renv` considered for an R environment lockfile.

An initial sample join was produced with a small Python script. For the final app, we plan to consolidate the core preparation workflow in R to simplify execution and maintenance.

8. Product

This is tentative, but we envision that the final product will be a deployed, patient-facing **Shiny hospital comparison app**, accompanied by a shared repository containing acquisition code, preparation functions, documentation, and the final Quarto report.

A user will select a condition and then narrow hospitals by available location fields, such as state or city. The app will display condition-specific mortality and readmission measures alongside hospital characteristics and overall stars. Users will be able to sort the table and select hospitals for a closer comparison. Definitions, missing-value explanations, and measurement dates will be accessible within the interface.

Historical payments may be included in a separate optional view if the team can present them clearly without implying current affordability. An initial release will not depend on this feature. The app will not claim to account for insurance coverage, individual treatment suitability, or all dimensions of hospital quality.

The repository will document how to install dependencies, refresh data, and start the app using `shiny::runApp()`. The deployed URL will be added once hosting is established. Cached data will keep the interface responsive and allow it to function independently of temporary API outages.

9. Timeline and feasibility

Period	Planned milestone
November 11–13	Finalize the proposal, confirm scope, and retain evidence of API access
November 16–20	Meet with an instructor or TA and revise the plan based on feedback

Period	Planned milestone
November 21–30	Complete acquisition, cleaning, provenance, and data validation
December 1–10	Build the Shiny comparison interface and explanatory graphics
December 11–16	Check usability and correctness, deploy, and prepare documentation and slides
December 17	Present the project in class

The exact deadlines for slides, the final write-up, and repository submission will be confirmed on CoursePlus.

The **minimum viable product** is a Maryland app with three condition choices, a hospital comparison table, a selected-hospital comparison view, and clear definitions, dates, and missingness indicators. We have demonstrated API access, so the main remaining challenges are interpretation, measure alignment, usability, and reliable deployment.

We will manage scope by prioritizing the three-condition comparison over a national rollout, geographic distance calculations, composite scores, or historical payment visualizations. If data completeness limits a comparison, we will show that limitation rather than impute a hospital's performance. If hosting presents difficulties, local execution will remain available while we resolve deployment. Further extensions will be considered only after the core workflow is working and reproducible.

10. Team Responsibilities

Sankalp (Sunny) Kumar: Confirm the initial measures. Wanting Shan: Build a reproducible data pipeline. Shiyi Zhang: Develop transparent comparisons. Mason Cleveland: Build the Shiny interface. Yutong (Benny) Li: Verify and deploy the product.

We will coordinate through regular meetings and use GitHub to share code, track progress, and collaborate on the project.